

ACTIVIDADES DE PREPARACIÓN DE ENTORNO

Definición de la infraestructura como código mediante el archivo docker-compose.yml.

En esta etapa, se configura un servicio contenedorizado para la base de datos

PostgreSQL (versión 16 basada en Alpine Linux), estableciendo un mapeo de puertos (5432) y definiendo volúmenes persistentes (pgdata) para garantizar la persistencia e integridad de la información ante reinicios del contenedor.

```
lfgelo@lfgelopc in ~/COMPARTIDO/PERSON/UIDE/BISNESS_INTELLENGI/PRACT_ONE
) bat docker-compose.yml

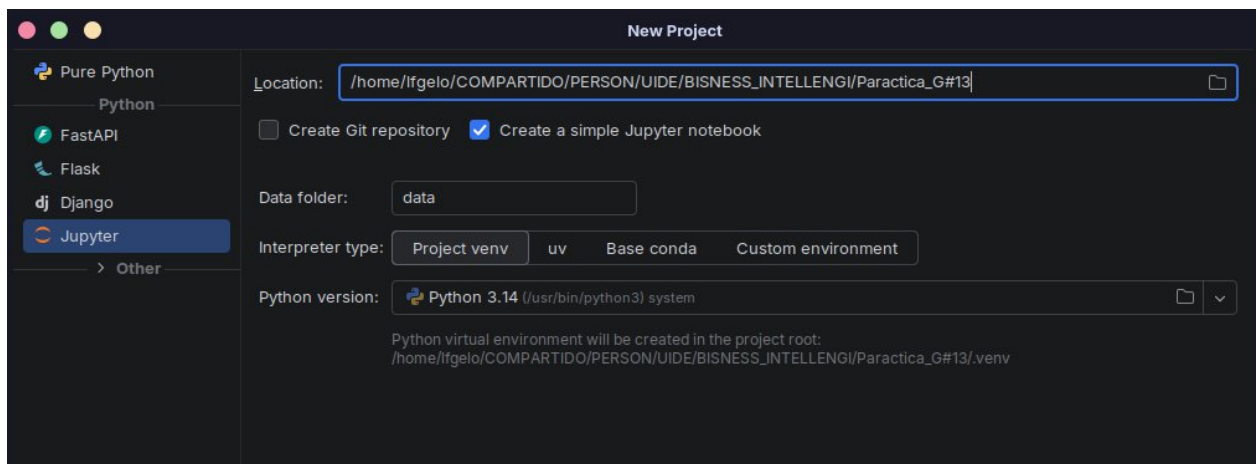
File: docker-compose.yml

1  services:
2    postgres_bi:
3      image: postgres:16-alpine
4      container_name: docker_postgres
5      restart: always
6      environment:
7        POSTGRES_USER: ${POSTGRES_USER}
8        POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
9        POSTGRES_DB: ${POSTGRES_DB}
10     ports:
11       - "${POSTGRES_PORT}:5432"
12     volumes:
13       - pgdata:/var/lib/postgresql/data
14
15  volumes:
16    pgdata:
```

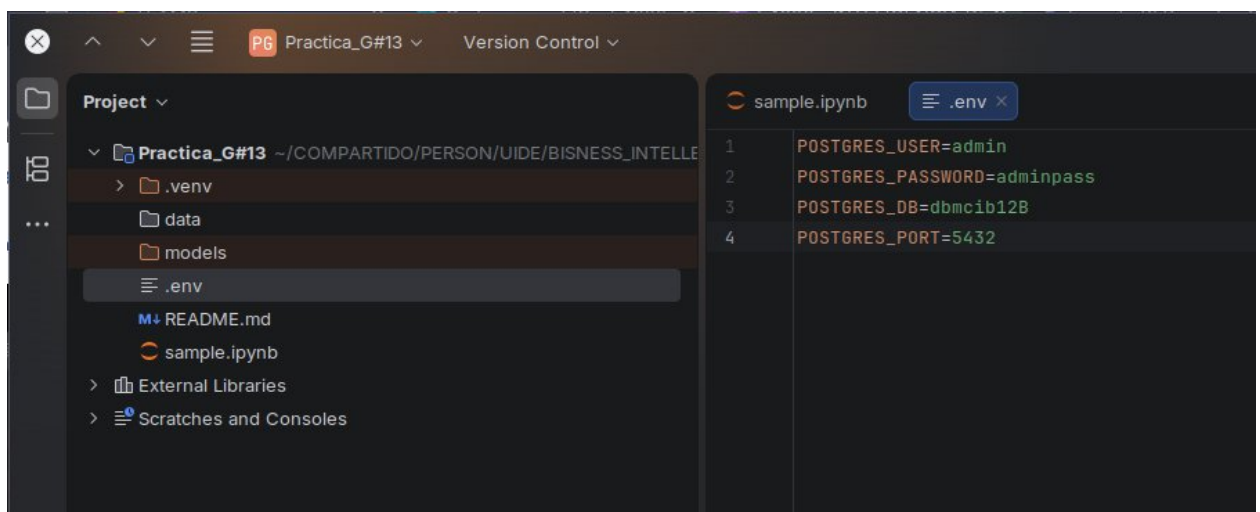
Verificación del despliegue exitoso del entorno de base de datos. Mediante la ejecución del comando docker ps en la terminal, se audita el estado del contenedor docker_postgres, confirmando que el servicio se encuentra activo (Up) y el puerto de comunicación está expuesto correctamente para la ingesta de datos.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a468dbce62aa	postgres:16-alpine	"docker-entrypoint.s..."	33 hours ago	Up 14 seconds	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	docker_postgres

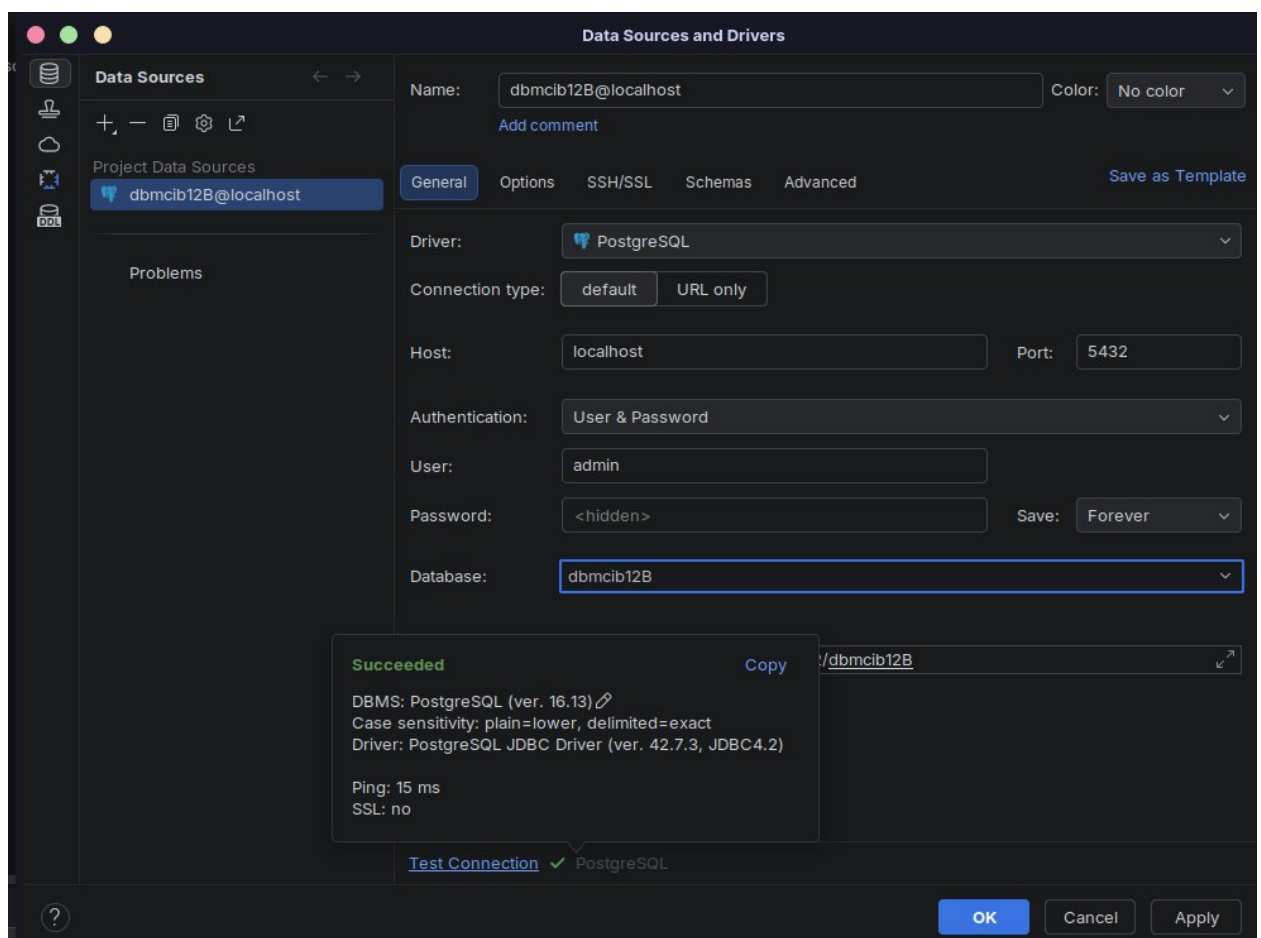
Inicialización del entorno de desarrollo integrado (IDE). Se crea un nuevo proyecto estructurado para análisis de datos, configurando un entorno virtual aislado (.venv) con Python 3.14 y habilitando el soporte para Jupyter Notebooks. Esto asegura que las dependencias del proyecto no interfieran con el sistema operativo base.



Implementación de buenas prácticas de seguridad en la gestión de credenciales. Se configura un archivo .env en la raíz del proyecto para centralizar las variables de entorno críticas (usuario, contraseña, nombre de la base de datos y puerto), aislando esta información sensible del código fuente.



Validación de la conectividad entre el entorno de desarrollo y la instancia de PostgreSQL alojada en Docker. Se configura un Data Source en el IDE utilizando los parámetros de seguridad previamente establecidos, obteniendo una respuesta exitosa (Succeeded) que confirma la disponibilidad de la base de datos para las futuras operaciones de lectura y escritura.



Preparación del entorno de ejecución en Jupyter Notebook mediante la instalación de las dependencias requeridas para el proceso ETL. A través del gestor de paquetes pip, se aprovisionan las librerías fundamentales: pandas para el modelado y manipulación de DataFrames, junto con sqlalchemy y psycopg2-binary para establecer los motores de conexión e interactuar de forma transaccional con la base de datos PostgreSQL.

```
1 pip install pandas sqlalchemy psycopg2-binary
✓ [1] 2s 501ms

Requirement already satisfied: pandas in ./venv/lib/python3.14/site-packages (3.0.3)
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.50-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64
    .manylinux_2_28_x86_64.whl.metadata (9.5 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.12-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64
    .whl.metadata (4.9 kB)
Requirement already satisfied: numpy>=2.3.3 in ./venv/lib/python3.14/site-packages (from
pandas) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in ./venv/lib/python3
.14/site-packages (from pandas) (2.9.0.post0)
Collecting greenlet>=1 (from sqlalchemy)
  Downloading greenlet-3.5.1-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
```

[Code](#) [Markdown](#) [SQL](#)

Luis Felipe Guamán León
Estudiante